

Servicio Nacional de Aprendizaje - SENA

Guías Bases de Datos

Bases de Datos SQL y NoSQL

Joel Andrés Mendoza Buriticá

Ficha: 3169892

Tecnólogo en Análisis y Desarrollo de Software

Julio de 2025

Tabla de contenido

Introducción	5
Importante	5
Guía AA00	6
Consulta	6
¿Qué es un dato?	6
¿Qué es información?	6
¿Cómo se puede garantizar la disponibilidad de los datos en la empresa?	6
¿A qué nos referimos con confiabilidad de la información?	6
¿Por qué consideras que la integridad de la información es esencial?	6
¿Qué tanto puede impactar una falla de autenticidad en los datos de una organización?	7
Sistemas Numéricos	7
Datos en lenguaje de programación.....	8
¿Cuál es la estructura de un registro de memoria?	8
¿Cómo se crea un puntero en los lenguajes de programación?	8
¿Qué es un sistema de archivos y cuáles son los más utilizados?	8
¿Cómo se almacenan los datos en un disco duro?	8
Tipos de datos	8
¿Cuáles son los tipos de dato que existen en lenguajes de programación?	8
¿Cuáles son los tipos de dato que existen en las bases de datos?	9
¿Qué es un Collation en base de datos?	9
Guía AA01	10
Consulta	10
¿Cuáles son los componentes de una base de datos?	10
¿Qué tipos de base de datos existen?	10
¿Qué programas permiten gestionar una base de datos?	12
¿Cuáles son los tipos de datos que pueden ser almacenados en una base de datos?	12
Grupo de tablas de datos	12
Guía AA02	14
Consulta	14
¿Qué es un diagrama?	14
¿Qué tipos de diagramas conoce o ha utilizado?	14

¿Cómo se estructura un diagrama?	14
¿Cómo se construye un diagrama?	14
¿Cuál es el objetivo de usar diagramas?	14
¿Por qué consideras que deben usarse diagramas en el diseño de soluciones de software?	15
UML	15
¿Cuáles son los tipos de diagrama más relevantes en UML? Describa al menos 4 de ellos.	15
¿Cómo representar instancias de las clases en UML? Elabore un ejemplo de una clase. Con sus atributos y características.	15
¿UML permite incluir notas, ¿cómo y para qué?	16
¿Qué es un método constructor y para qué se utiliza?	16
¿Qué tipos de Relaciones pueden especificarse entre clases?	16
Diagramas	16
¿Cómo se crea un Diagrama de Casos de Uso en UML?	16
Elementos que componen un Diagrama de Casos de Uso	17
Diagrama de uso	17
Base de datos (Diagrama entidad-relación)	18
Base de datos en SQL (Xampp):	18
Base de datos en NoSQL (MongoDB)	21
Dashboard MongoDB	22
GUIA A003	24
Consulta	24
¿Qué es un repositorio en Git y cuál es su función dentro de un proyecto de desarrollo?	24
¿Qué problema resuelve el uso de sistemas de control de versiones como Git en proyectos de desarrollo de software?	24
¿Cómo permite Git la colaboración entre varios desarrolladores sin que sus cambios interfieran entre sí?	24
¿Cuál es la diferencia entre un commit y un push en el flujo de trabajo de Git?	24
¿Qué es un pull request en GitHub y cómo ayuda a revisar y fusionar los cambios de los colaboradores?	25
¿Qué ventajas ofrece el uso de ramas (branches) en Git cuando se está desarrollando una nueva funcionalidad o corrigiendo un error?	25

¿En qué situaciones se recomienda usar un fork en GitHub?	25
Actividad 1: Configuración Inicial.....	26
Creación de cuenta Github	26
Creación Organización Github	26
Creación repositorio Github	27
Creación Proyecto en la organización	27
Actividad 2: Ramas.....	28
Actividad 3: Pull Request e Issues	30
Actividad 4: Actions.....	33
Explicación funcionamiento Token	33
Continuación código .yml	34
Pagina funcionando en: https://jborg10.github.io/JBGit/	35
Referencias.....	36

Introducción

En este trabajo, se abordan temas de gran importancia relacionados con el diseño, análisis y gestión de bases de datos, en bases de tipo SQL y no SQL, y su aplicación práctica dentro del desarrollo de software que usamos en la carrera. Esto a través de diversas guías académicas, enfocadas en competencias del área, desde el entendimiento de datos y representación en los sistemas llegando hasta el modelado usando diagramas UML y la practica con herramientas como Git y GitHub para fortalecer los conocimientos y fomentar el trabajo en equipo. Además de ello, también se incluye una sección práctica donde se construye una base de datos con tablas basadas en un diagrama de clases, teniendo en cuenta sus campos, tipos de datos y restricciones, cosa de vital importancia para entender los diagramas que se ven dentro de esta carrera y crear correctamente bases de datos funcionales y que cumplan con las condiciones estipuladas aparte de servir de constancia de la obtención de esta información en sí.

Importante

En este documento se encuentran la información requerida de las guías AA00, AA01, AA02 y AA03 respectivamente, organizadas en el mismo documento con la información clasificada como se puede ver en la tabla de contenido.

Cada guía está nombrada dentro del archivo con el nombre de la guía (Ejemplo: GUIA AA00) separada por las secciones especificadas en cada guía, decidí hacerlo de esta forma para tener toda la información en un mismo archivo y, así poder editar o cambiar información en caso de ser necesario hacerla (Como en el caso en el que debía volver a una guía anterior para crear las tablas basado en el diagrama).

Espero esto también facilite para usted la lectura y que le permita entender de mejor forma la estructura del documento.

Guía AA00

Consulta

¿Qué es un dato?

Un dato es una cosa existente, un tipo de cualquier cosa que puede ser clasificada, por sí sola no significa mucho pero cada dato es importante para obtener información necesaria.

¿Qué es información?

La información es un conjunto de datos que están organizados y procesados, básicamente se puede decir que tienen sentido y valor para quien trabaja con ellos, gracias a ello podemos tomar decisiones o entender una situación cosa que es vital para la evolución de algún tipo de consulta o investigación.

¿Cómo se puede garantizar la disponibilidad de los datos en la empresa?

Lo podríamos hacer con respaldos constantes, teniendo sistemas de almacenamiento redundantes, hacer lo posible por tener servidores seguros y facilitar la forma en la que las personas con acceso a esta información pueden visualizarla para asegurar que los datos estén siempre accesibles cuando se necesiten.

¿A qué nos referimos con confiabilidad de la información?

A que los datos sean precisos, coherentes y verídicos, lo que nos permite confiar en que la información es real, evitar errores y retrasos innecesarios.

¿Por qué consideras que la integridad de la información es esencial?

Porque nos asegura que los datos no han sido modificados sin permiso. Ya que, si la integridad se compromete, las decisiones que están basadas en esa información pueden ser incorrectas y perjudiciales.

¿Qué tanto puede impactar una falla de autenticidad en los datos de una organización?

Puede generar consecuencias muy graves desde la pérdida de credibilidad, decisiones incorrectas, fraudes, retrasos de entrega hasta incluso sanciones legales dependiendo lo grave del caso, pero en todas estas la empresa tendría pérdidas tanto económicas como la misma credibilidad de la compañía.

Sistemas Numéricos

Decimal	Binario	Octal	Hexadecimal
5050	1001110111010	11672	13BA
4096	1000000000000	10000	1000
4567	1000111010111	10727	11D7
3800	111011011000	7330	ED8
1098	10001001010	2112	44A
1024	10000000000	2000	400
2048	100000000000	4000	800
2512	100111010000	4720	9D0
1970	11110110010	3662	7B2
680	1010101000	1250	2A8
512	1000000000	1000	200
300	100101100	454	12C
256	100000000	400	100
128	10000000	200	80
190	10111110	276	BE
64	1000000	100	40
56	111000	70	38
10	1010	12	A
780	1100001100	1414	30C
912	1110010000	1620	390
360	101101000	550	168
556	1000101100	1054	22C
428	110101100	654	1AC
990	1111011110	1736	3DE
604	1001011100	1134	25C
506	111111010	772	1FA
100	1100100	144	64

Datos en lenguaje de programación

¿Cuál es la estructura de un registro de memoria?

Incluye el nombre del registro, el tamaño (en bits), y el tipo de dato que almacena.

¿Cómo se crea un puntero en los lenguajes de programación?

En C: `int a = 10; int *ptr = &a;`

En Java: `Caja puntero = new Caja(); puntero.valor = 10;`

¿Qué es un sistema de archivos y cuáles son los más utilizados?

Organiza cómo se guardan y recuperan los datos en un dispositivo (como un disco).

Define cómo se nombran, almacenan y acceden los archivos.

Algunos de los más usados son:

- FAT32: común en memorias USB.
- NTFS: usado por Windows.
- EXT4: típico en Linux.
- APFS: usado por macOS.

¿Cómo se almacenan los datos en un disco duro?

Se almacenan en forma magnética sobre platos giratorios. Cada plato tiene pistas y sectores, donde se escriben los datos en forma de campos magnéticos que representan ceros y unos. Un brazo mecánico con un cabezal lee o escribe los datos cuando el plato gira.

Tipos de datos

¿Cuáles son los tipos de dato que existen en lenguajes de programación?

Enteros (int): Números sin decimales

Flotantes (float, double): Números con decimales

Caracteres (char): Un solo carácter

Cadenas (string): Conjunto de caracteres

Booleanos (bool): Verdadero o falso

Listas o arreglos: Conjuntos de datos ordenados

Objetos (object): Instancias de clases

Nulos (null): Valor sin contenido

¿Cuáles son los tipos de dato que existen en las bases de datos?

Numéricos: INT, BIGINT, FLOAT, DECIMAL

Texto: CHAR, VARCHAR, TEXT

Fecha y hora: DATE, TIME, DATETIME, TIMESTAMP

Booleanos: BOOLEAN, BIT

Binarios: BLOB, BINARY

¿Qué es un Collation en base de datos?

Son reglas que definen cómo se comparan y ordenan los textos en una base de datos puede ser más o menos específico teniendo en cuenta como se configure la base de datos para la verificación de los datos que la conforman. Incluye:

- El idioma/localización (ej. español, inglés)
- Si distingue mayúsculas/minúsculas (case-sensitive)
- Si distingue acentos (accent-sensitive)

Guía AA01

Consulta

¿Cuáles son los componentes de una base de datos?

- Tablas: Para guardar datos.
- Consultas: Para acceder a la información contenida en una base de datos. Con las consultas se pueden modificar, agregar, eliminar y mostrar datos.
- Informes: Para presentar información contenida en una base de datos. Esta información se muestra de una forma en concreto y en un determinado formato para que pueda ser imprimida.
- Formularios: Para ingresar datos.
- Macros: Para crear instrucciones y ejecutar funciones.

¿Qué tipos de base de datos existen?

- Macros: Para crear instrucciones y ejecutar funciones.
- Base de datos estática: Es de sólo lectura. Es decir, es la base que no permite que sus datos sean modificados o eliminados una vez que son almacenados.
- Base de datos dinámica: Es la que permite que sus datos sean modificados o eliminados una vez que son almacenados.
- Base de datos relacional: Es la colección de datos que es organizada utilizando tablas descriptivas. Estas bases utilizan SQL, un lenguaje de consulta estructurada para gestionar y consultar datos.

- Base de datos no relacional: Es la colección que se diferencia de la base relacional por no utilizar el lenguaje SQL como herramienta principal, sino que da uso del lenguaje denominado NoSQL. Además, tampoco utilizan tablas para estructurar su información.
- Base de datos bibliográfica: Es la que guarda metadatos bibliográficos de materiales como libros, artículos, periódicos, revistas, tesis, patentes y conferencias. Es decir, son datos que brindan información bibliográfica, pero no el contenido en sí (la información completa). Ejemplo de ello sería nombres de autores, títulos, resúmenes, índices o palabras claves. Estas bases de datos pueden ser en formato físico o digital.
- Base de datos de texto completo: Es la que almacena el contenido completo de libros, artículos, diarios, disertaciones, periódicos, revistas, tesis y otros documentos escritos.
- Base de datos orientada a objetos: Es aquella base en la cual los datos son representados como objetos.
- Base de datos jerárquica: Es la que almacena información de forma jerárquica. Es decir, es la colección que comprende una arquitectura del tipo arbórea, en la cual un nodo se ramifica en otros nodos, y estos se dividen en otros.
- Base de datos de red: Es aquella en la que los datos contenidos están conectados entre sí a través de ciertos enlaces.
- Base de datos transaccional: Es la que permite llevar a cabo transacciones cortas de carácter digital a gran velocidad.
- Base de datos multidimensional: Al igual que la base de datos relacional, este tipo de base almacena la información utilizando tablas, la diferencia se encuentra en su forma estructural, que es organizada en una especie de cubo de información.

- Base de datos deductiva: Es el banco que permite establecer conclusiones partiendo de ciertas premisas.

- Base de datos documental: Es la base que ordena la información de forma semiestructurada utilizando archivos como JSON y XML.

¿Qué programas permiten gestionar una base de datos?

Relacionales: MySQL, PostgreSQL, Oracle. No relacionales: MongoDB. Herramientas: phpMyAdmin, DBeaver, MySQL Workbench.

¿Cuáles son los tipos de datos que pueden ser almacenados en una base de datos?

Númericos exactos y aproximados, fecha y hora, caracteres, binarios, y tipos especiales como JSON, XML, geography, etc.

Grupo de tablas de datos

Campo	Tipo de dato	Longitud
id_usuario	INT	
nombre	VARCHAR	100
correo	VARCHAR	100
tipo_usuario	VARCHAR	20
fecha_registro	DATE	

Campo	Tipo de dato	Longitud
id_categoria	INT	
nombre	VARCHAR	50

Campo	Tipo de dato	Longitud
id_libro	INT	
titulo	VARCHAR	150
autor	VARCHAR	100
id_categoria	INT	
stock	INT	
año_publicacion	INT	

Campo	Tipo de dato	Longitud
id_prestamo	INT	
id_usuario	INT	
id_libro	INT	
fecha_prestamo	DATE	
fecha_devolucion	DATE	
estado	VARCHAR	20

Campo	Tipo de dato	Longitud
id_historial	INT	
id_usuario	INT	
id_libro	INT	
fecha_evento	DATETIME	
tipo_evento	VARCHAR	50

Guía AA02

Consulta

¿Qué es un diagrama?

Un diagrama es una representación gráfica en la que se muestra información, procesos, relaciones o estructuras de forma visual. Facilita la comprensión de los conceptos complejos usando símbolos, formas y conexiones. Todo esto es de gran ayuda para la creación de una base de datos ya que sirve de maqueta y hace que el proceso de creación de la misma sea más corto y eficaz.

¿Qué tipos de diagramas conoce o ha utilizado?

Por ahora he usado diagramas de red, de entidad-relación (ER), y mapas mentales más que todo los de entidad relación ya que es el que usualmente me sirve de maquetación para la creación de las bases de datos en SQL.

¿Cómo se estructura un diagrama?

Se estructura con elementos clave como símbolos o figuras (círculos, rectángulos, flechas), títulos, y una disposición lógica de los elementos que refleje el contenido o proceso para hacer que este sea fácil de comprender para mí mismo y las personas que vean estos diagramas.

¿Cómo se construye un diagrama?

Lo importante es primero definir un propósito para luego recopilar la información que sea necesaria para hacerlo. Después de esto, se eligen los símbolos adecuados y se organiza la información de una forma clara esto se puede hacer usando herramientas como aplicaciones o en su defecto hacerlo a mano.

¿Cuál es el objetivo de usar diagramas?

Simplificar y aclarar ideas, procesos o estructuras, permitiendo una comunicación más efectiva y rápida comprensión entre un grupo de personas.

¿Por qué consideras que deben usarse diagramas en el diseño de soluciones de software?

Porque ayudan a visualizar la arquitectura, como se manejan los datos y la lógica del sistema. Gracias a esto se facilita la comunicación entre el equipo y les permiten detectar errores o mejoras antes de codificar evitando pérdidas de tiempo innecesarias.

UML

¿Cuáles son los tipos de diagrama más relevantes en UML? Describa al menos 4 de ellos.

Diagrama de Clases: Representa las clases, atributos, métodos y relaciones entre ellas.

Diagrama de Casos de Uso: Muestra las funciones del sistema desde la perspectiva del usuario.

Diagrama de Secuencia: Describe la interacción entre objetos en orden temporal.

Diagrama de Actividades: Representa flujos de trabajo o procesos con decisiones y ciclos.

¿Cómo representar instancias de las clases en UML? Elabore un ejemplo de una clase. Con sus atributos y características.

Se puede hacer de la siguiente forma:

Clase:

Persona: nombre: String

edad: int

saludar(): void

Instancia: (:Persona)

nombre = "Juan"

edad = 18

¿UML permite incluir notas, ¿cómo y para qué?

Sí, UML permite agregar notas usando un símbolo de rectángulo que hay en el programa. Se usa para aclarar decisiones, agregar comentarios o justificar elementos del modelo que se está elaborando.

¿Qué es un método constructor y para qué se utiliza?

Es un método que se ejecuta al crear un objeto de una clase. Lo utilizamos para inicializar atributos y garantizar que el objeto comience con valores correctos para el proceso que se necesita.

¿Qué tipos de Relaciones pueden especificarse entre clases?

Asociación: Relación simple entre clases.

Herencia (Generalización): Una clase hija hereda de una clase considerada “padre”

Agregación: Una clase contiene a otra, pero ambas pueden existir por separado.

Diagramas

¿Cómo se crea un Diagrama de Casos de Uso en UML?

Para crear un Diagrama de Casos de Uso en UML, se deben seguir estos pasos:

1. Identificar los actores: Personas, sistemas u objetos externos que interactúan con el sistema.
2. Definir los casos de uso: Funciones o servicios que el sistema ofrece a los actores.
3. Establecer las relaciones: Conexiones entre actores y casos de uso (comunicación, inclusión, extensión).

4. Organizar el diagrama: Colocar el sistema dentro de un rectángulo y distribuir los elementos de manera clara.

Elementos que componen un Diagrama de Casos de Uso

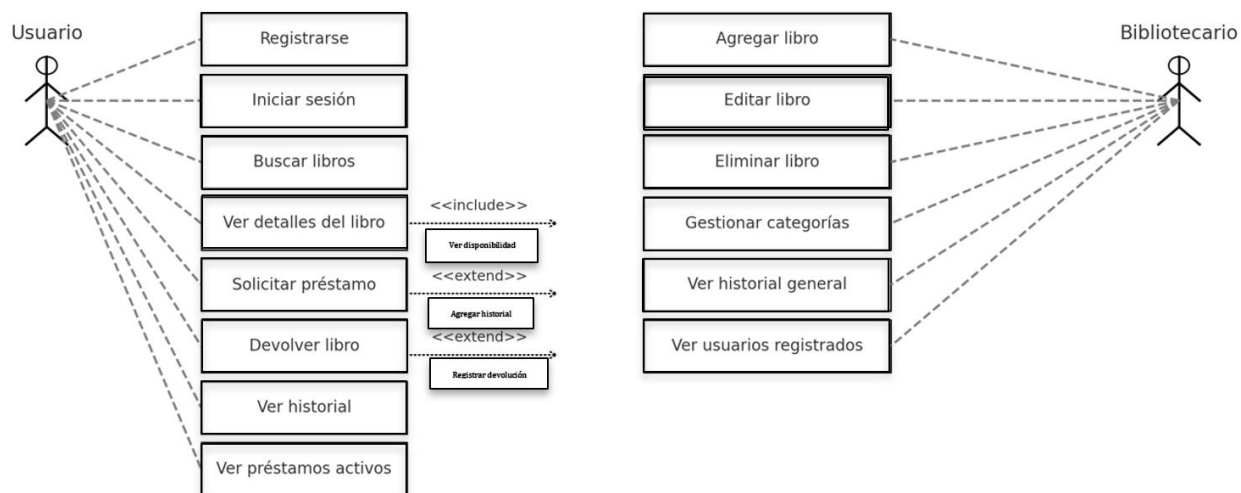
- Actores: Representados con una figura de persona.
- Casos de Uso: Óvalos que describen funcionalidades.
- Sistema: Un rectángulo que encierra los casos de uso.
- Relaciones:

Línea simple (asociación)

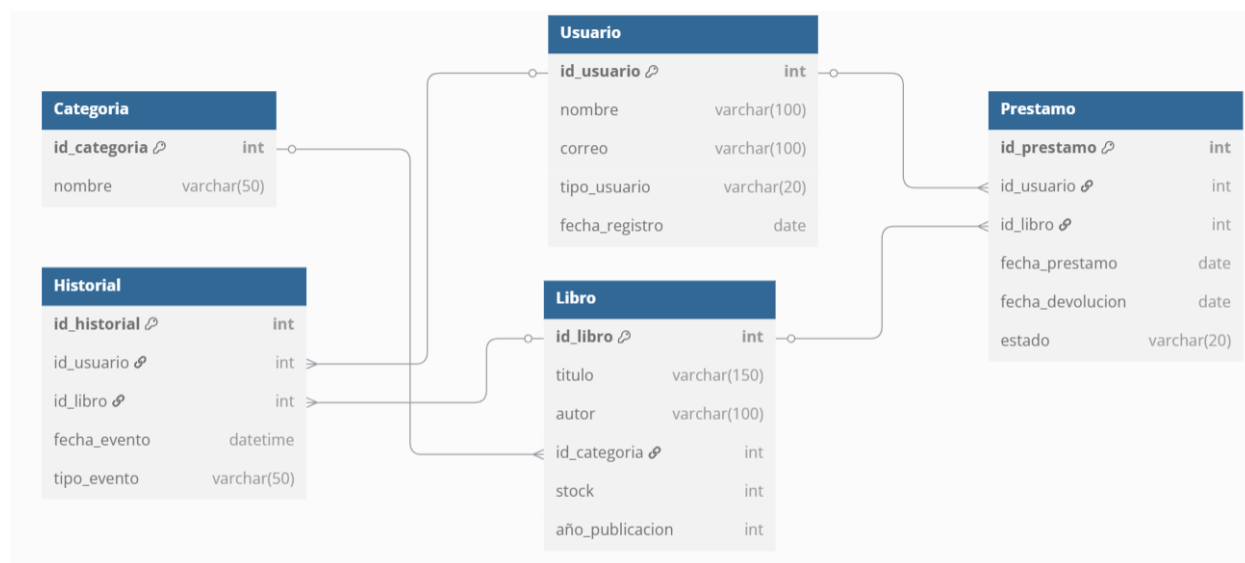
<<include>> (uso obligatorio de otro caso)

<<extend>> (uso opcional o condicional)

Diagrama de uso



Base de datos (Diagrama entidad-relación)



Base de datos en SQL (Xampp):

```

use database biblioteca

CREATE TABLE Categoria (
  id_categoria INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50)
);

CREATE TABLE Usuario (
  id_usuario INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100),
  correo VARCHAR(100),
  tipo_usuario VARCHAR(20),
  fecha_registro DATE
);
  
```

```

CREATE TABLE Libro (
    id_libro INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150),
    autor VARCHAR(100),
    id_categoria INT,
    stock INT,
    año_publicacion INT,
    FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria)
);

CREATE TABLE Prestamo (
    id_prestamo INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT,
    id_libro INT,
    fecha_prestamo DATE,

    fecha_devolucion DATE,
    estado VARCHAR(20),
    FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario),
    FOREIGN KEY (id_libro) REFERENCES Libro(id_libro)
);

CREATE TABLE Historial (
    id_historial INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT,
    id_libro INT,
    fecha_evento DATETIME,
    tipo_evento VARCHAR(50),
    FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario),
    FOREIGN KEY (id_libro) REFERENCES Libro(id_libro)
);

```

✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0005 segundos.)

```
CREATE TABLE Categoria ( id_categoria INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(50) );
```

[Editar en línea] [Editar] [Crear código PHP]

✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0005 segundos.)

```
CREATE TABLE Usuario ( id_usuario INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100), correo VARCHAR(100), tipo_usuario VARCHAR(20), fecha_registro DATE );
```

[Editar en línea] [Editar] [Crear código PHP]

✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0005 segundos.)

```
CREATE TABLE Libro ( id_libro INT AUTO_INCREMENT PRIMARY KEY, titulo VARCHAR(150), autor VARCHAR(100), id_categoria INT, stock INT, año_publicacion INT, FOREIGN KEY (id_categoria) REFERENCES Categoria(id_categoria) );
```

[Editar en línea] [Editar] [Crear código PHP]

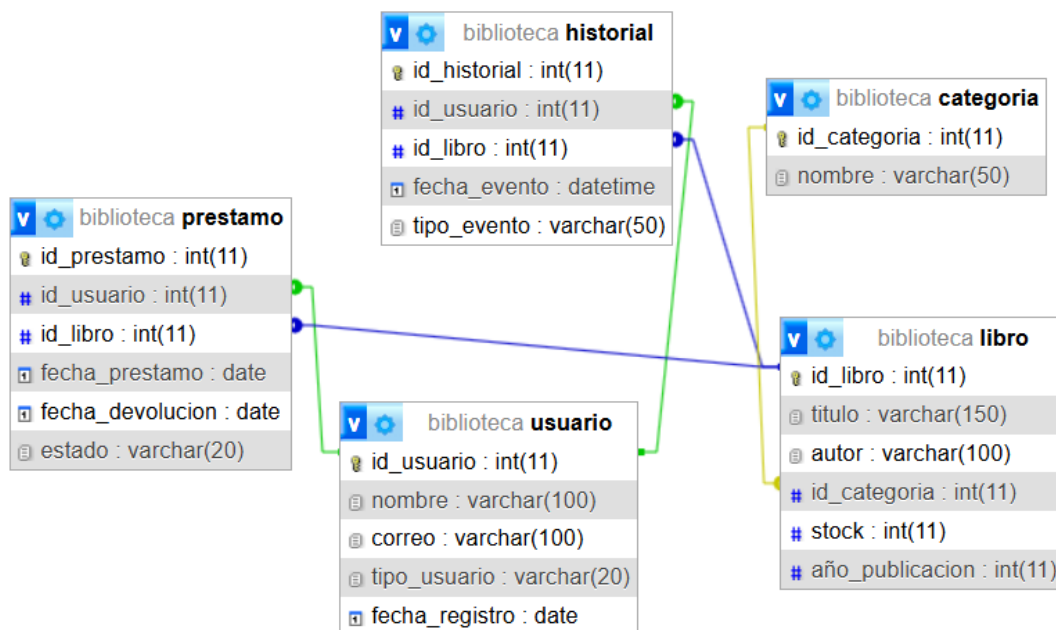
✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0004 segundos.)

```
CREATE TABLE Prestamo ( id_prestamo INT AUTO_INCREMENT PRIMARY KEY, id_usuario INT, id_libro INT, fecha_prestamo DATE, fecha_devolucion DATE, estado VARCHAR(20), FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario), FOREIGN KEY (id_libro) REFERENCES Libro(id_libro) );
```

[Editar en línea] [Editar] [Crear código PHP]

✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0004 segundos.)

```
CREATE TABLE Historial ( id_historial INT AUTO_INCREMENT PRIMARY KEY, id_usuario INT, id_libro INT, fecha_evento DATETIME, tipo_evento VARCHAR(50), FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario), FOREIGN KEY (id_libro) REFERENCES Libro(id_libro) );
```



Base de datos en NoSQL (MongoDB)

Insert Document

To collection Usuario

VIEW



```
1 ▼ {
2   "_id": { "$oid": "686ebd57b85783966d0e75c2" },
3   "nombre": "Joel",
4   "correo": "joel@gmail.com",
5   "tipo_usuario": "estudiante",
6   "fecha_registro": { "$date": "2025-01-15T00:00:00Z" }
7 }
8
```

Insert Document

To collection categoria

VIEW



```
1 ▼ { "_id": { "$oid": "686ebe26b85783966d0e75c5" }, |
2   "nombre": "Ciencia Ficción"
3 }
```



Insert Document

To collection Libro

VIEW



```
1  { "_id": {"$oid": "686ebef3b85783966d0e75c7"}, |
2    "titulo": "Cien años de soledad",
3    "autor": "Gabriel García Márquez",
4    "id_categoria": { "$oid": "686ebe26b85783966d0e75c5" },
5    "stock": 7,
6    "año_publicacion": 1967
7  }
```



Insert Document

To collection prestamo

VIEW



```
1  { "_id": {"$oid": "686ebfbbb85783966d0e75c9"},
2    "id_usuario": { "$oid": "686ebd57b85783966d0e75c2" },
3    "id_libro": { "$oid": "686ebf22b85783966d0e75c8" },
4    "fecha_prestamo": { "$date": "2025-07-01T00:00:00Z" },
5    "fecha_devolucion": { "$date": "2025-07-15T00:00:00Z" },
6    "estado": "devuelto"
7  }
```



Dashboard MongoDB

Libro

Enter a description

group	686ebe26b85...	686ebe79b85...	686ebef3b857...	686ebf22b857...	Cien años de ...
_id			1	1	
id_categoria	1	1			
titulo					1

Usuario

Enter a description

_id	correo	nombre	tipo_usuario
686ebd57b85783966d0e75c2	joel@gmail.com	Joel	estudiante
686ebd57b85783966d0e75c3	sebasg@gmail.com	Sebastian	estudiante
686ebd57b85783966d0e75c4	luisf@gmail.com	Luis Fernando	bibliotecario

Categorias

Enter a description

_id	nombre
686ebe26b85783966d0e75c5	Ciencia Ficción
686ebe79b85783966d0e75c6	Drama

Prestamo

Enter a description

_id	estado	id_libro	id_usuario
686ebfbbb85783966d0e75c9	devuelto	686ebf22b85783966d0e75c8	686ebd57b85783966d0e75c2
686ebfe0b85783966d0e75ca	pendiente	686ebef3b85783966d0e75c7	686ebd57b85783966d0e75c3

GUIA A003

Consulta

¿Qué es un repositorio en Git y cuál es su función dentro de un proyecto de desarrollo?

Es una estructura que almacena los cambios de un proyecto, desde sus archivos hasta incluso sus configuraciones. Permite control de versiones, facilitando el seguimiento de los cambios, la colaboración entre desarrolladores y la recuperación de versiones borradas.

¿Qué problema resuelve el uso de sistemas de control de versiones como Git en proyectos de desarrollo de software?

Git nos ayuda resolviendo varios problemas clave como la pérdida de información ya que guarda constantemente, juntar códigos de varios autores, saber quién, y cuando se cambió algo, se puede trabajar paralelamente sin problemas y facilita tener versiones de un mismo proyecto.

¿Cómo permite Git la colaboración entre varios desarrolladores sin que sus cambios interfieran entre sí?

Git nos permite a los desarrolladores trabajar en su propia rama (se le llama branch). Esto separa los cambios hasta que son revisados y se añaden (usando merge) en la rama principal. Así evitamos conflictos y nos aseguramos de que los cambios estén bien integrados sin afectar al resto del proyecto.

¿Cuál es la diferencia entre un commit y un push en el flujo de trabajo de Git?

Commit guarda los cambios de forma local en el repositorio del desarrollador mientras que Push envía estos cambios locales del commit al repositorio remoto para compartirlos con los otros desarrolladores.

¿Qué es un pull request en GitHub y cómo ayuda a revisar y fusionar los cambios de los colaboradores?

Es una solicitud para que los cambios hechos que hizo alguien en una rama sean revisados y, si funcionan correctamente, se añadan en la rama principal. Gracias a ello podemos revisar correctamente, comentar o sugerir a los mismos desarrolladores, realizar pruebas y asegurar una buena calidad de producto.

¿Qué ventajas ofrece el uso de ramas (branches) en Git cuando se está desarrollando una nueva funcionalidad o corrigiendo un error?

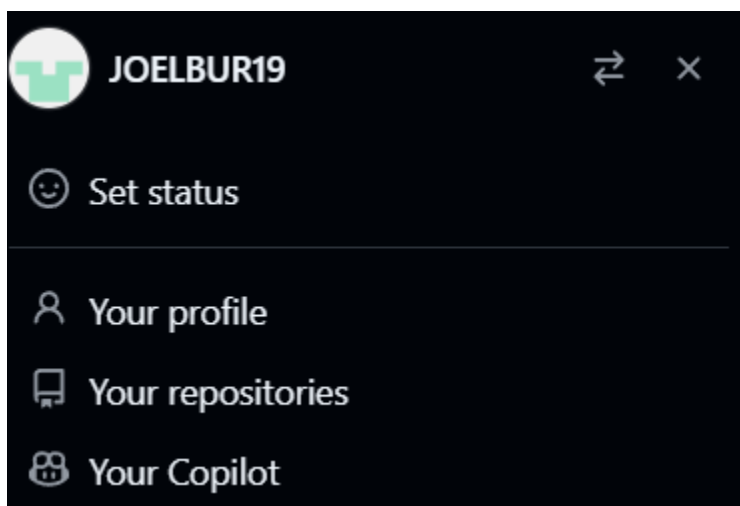
Evita que los cambios afecten al código principal mientras se está trabajando aparte. Además de por ello mismo manejar múltiples tareas al mismo tiempo. Nos ayuda también a reducir el riesgo de errores y a descartar ramas que no sirvan o incluso traer versiones pasadas de vuelta.

¿En qué situaciones se recomienda usar un fork en GitHub?

El fork se usa principalmente cuando alguien quiere contribuir a un proyecto del que no hace parte directamente creando una copia de ese repositorio a tu cuenta y trabajar en una copia personal que puedes pedir añadir usando el pull request anteriormente mencionado. Se recomienda usarlo cuando el código que quieres aportar es de código abierto, no contamos con permiso para editar el original y cuando queremos hacer alguna prueba.

Actividad 1: Configuración Inicial

Creación de cuenta Github



Creación Organización Github

Tell us about your organization

Set up your organization

Organization name *

JBOrg10

This will be the name of your account on GitHub.
Your URL will be: <https://github.com/JBOrg10>.

Contact email *

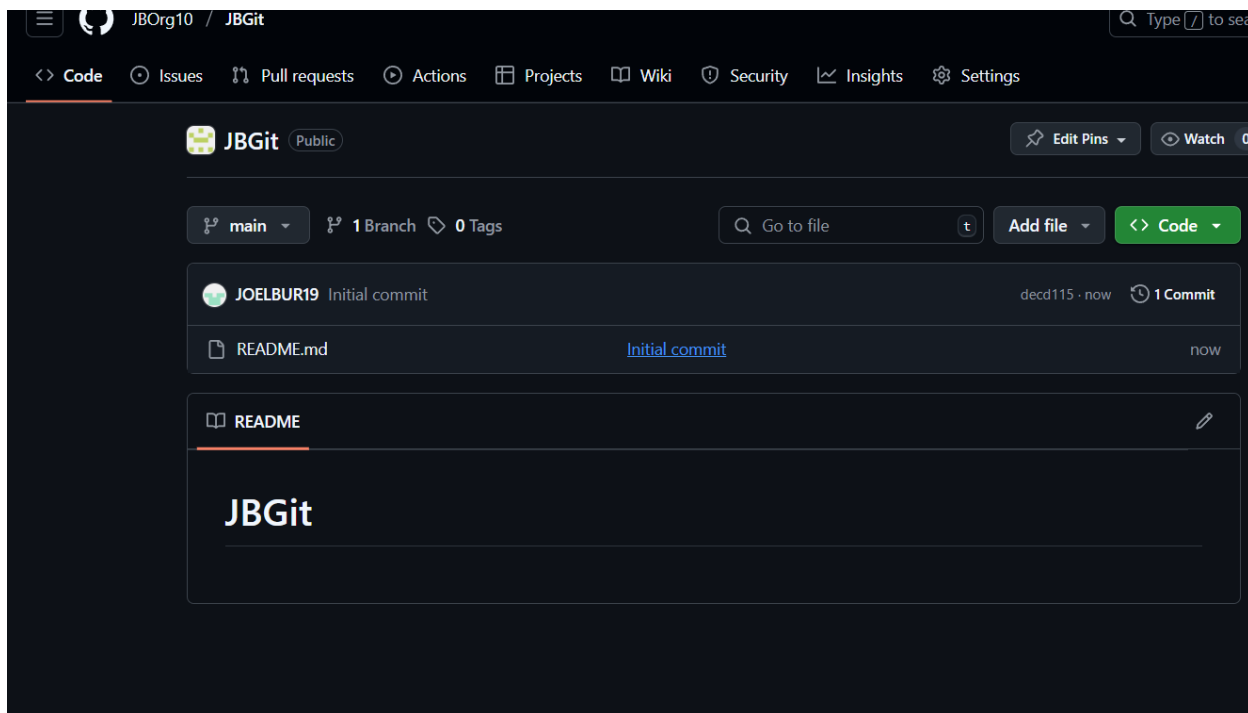
JBOrg10@contacto.com

This organization belongs to:

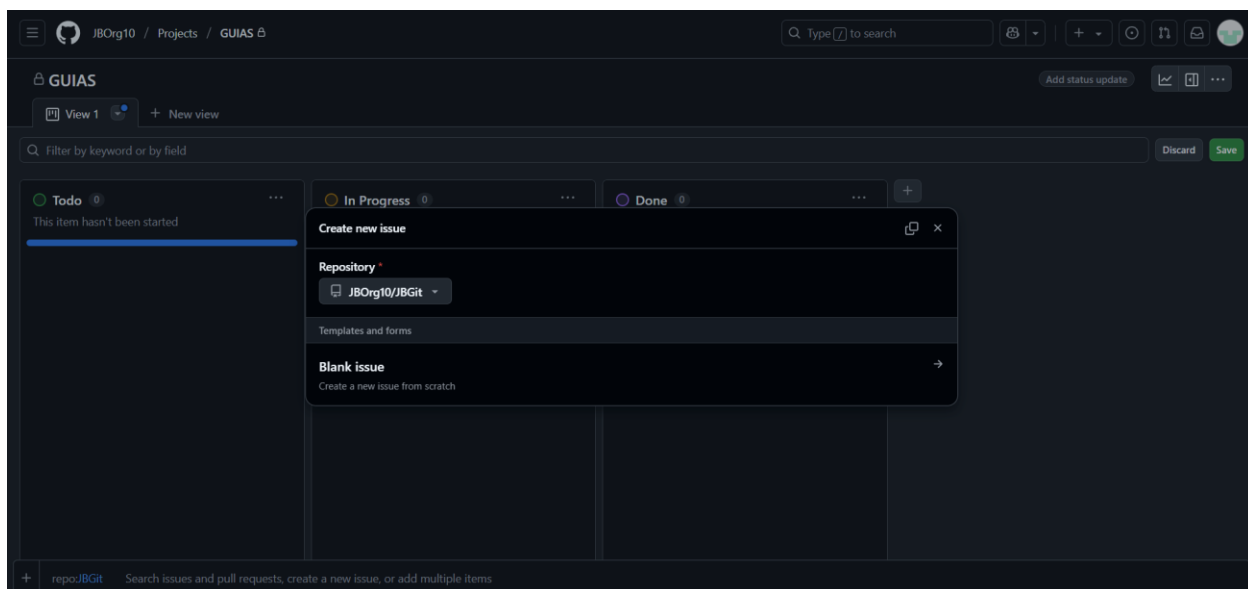
☒ **My personal account**
I.e., JOELBUR19

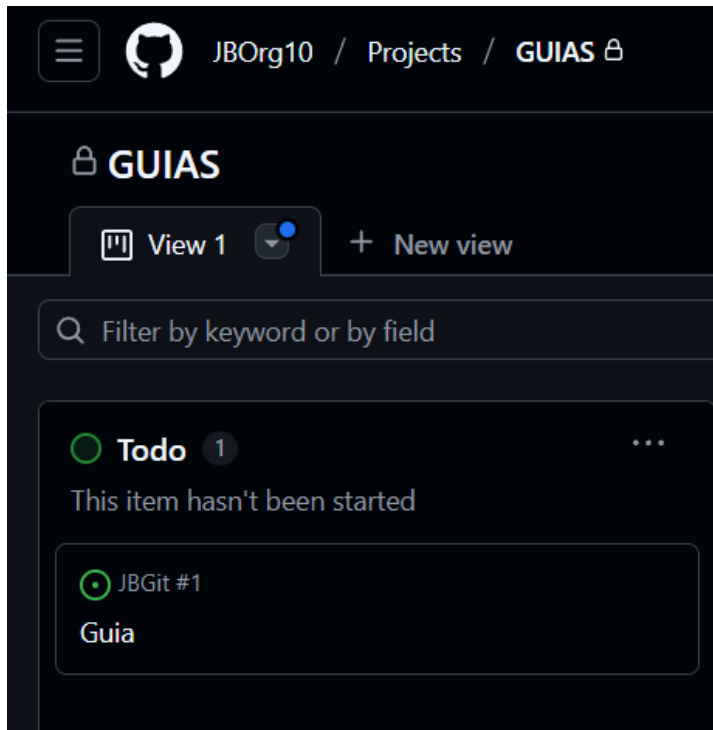
☐ **A business or institution**
For example: GitHub, Inc., Example Institute, American Red Cross

Creación repositorio Github



Creación Proyecto en la organización





Actividad 2: Ramas

```
Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (main)
$ git checkout -b diseño-página
Switched to a new branch 'diseño-página'

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (diseño-página)
$ git status
On branch diseño-página
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html
        modified:   styles.css

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        LICENSE.txt
        REQUIREMENTS.md
        backlog.md

no changes added to commit (use "git add" and/or "git commit -a")
```

```
Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (diseño-página)
$ git add -A

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (diseño-página)
$ git commit -m "Modificación en Rama Pagina"
[diseño-página ae27dcc] Modificación en Rama Pagina
5 files changed, 14 insertions(+), 1 deletion(-)
create mode 100644 LICENSE.txt
create mode 100644 REQUIREMENTS.md
create mode 100644 backlog.md
```

```
Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (diseño-página)
$ git push -u origin diseño-página
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 794 bytes | 794.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'diseño-página' on GitHub by visiting:
remote:   https://github.com/JBOrg10/JBGit/pull/new/dise%C3%B1o-p%C3%A1gina
remote:
To https://github.com/JBOrg10/JBGit.git
 * [new branch]      diseño-página -> diseño-página
branch 'diseño-página' set up to track 'origin/diseño-página'.

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (diseño-página)
$ git status
On branch diseño-página
Your branch is up to date with 'origin/diseño-página'.

nothing to commit, working tree clean
```

Modificación en Rama Pagina #2

[Open](#)
JOELBUR19 wants to merge 1 commit into `main` from `diseño-página`

⚠ The head ref may contain hidden characters: `"dise\u00F1o-p\u00E1gina"`

Conversation 0

Commits 1

Checks 0

Files changed 5

JOELBUR19 commented now

Member

...

Actualización desde la rama

Modificación en Rama Pagina

ae27dcc

No conflicts with base branch

Merging can be performed automatically.

Merge pull request

You can also merge this with the command line. [View command line instructions.](#)

Reviewers

No reviews

Still in progress? [Convert to draft](#)

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet



Pull request successfully merged and closed

You're all set — the `diseño-página` branch can be safely deleted.

[Delete branch](#)



JOELBUR19

Merge pull request #2 from JBOrg10/diseño-página

7f167b0 · now

 6 Commits

 LICENSE.txt	Modificación en Rama Pagina	10 minutes ago
 README.md	Update README.md	4 days ago
 REQUIREMENTS.md	Modificación en Rama Pagina	10 minutes ago
 backlog.md	Modificación en Rama Pagina	10 minutes ago
 index.html	Modificación en Rama Pagina	10 minutes ago
 styles.css	Modificación en Rama Pagina	10 minutes ago

Actividad 3: Pull Request e Issues

Añadir carpetas faltantes #3

 Open

 **JOELBUR19** opened now · edited by JOELBUR19

Edits ▼ Member ...

Carpetas no aparecen en la rama principal

[Create sub-issue](#) ▼ 

```

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (main)
$ git checkout -b carpetas-error
Switched to a new branch 'carpetas-error'

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (carpetas-error)
$ git add -A

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (carpetas-error)
$ git status
On branch carpetas-error
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   assets/nada.txt.txt
    new file:   documentation/nada.txt.txt
    modified:   index.html
    new file:   src/nada.txt.txt

```

```

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (carpetas-error)
$ git commit -m "Solucion carpetas y HTML"
[carpetas-error 9f53a61] Solucion carpetas y HTML
 4 files changed, 4 insertions(+), 1 deletion(-)
 create mode 100644 assets/nada.txt.txt
 create mode 100644 documentation/nada.txt.txt
 create mode 100644 src/nada.txt.txt

Joel@Joel MINGW64 ~/Desktop/Act2/JBGit (carpetas-error)
$ git push -u origin carpetas-error
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 645 bytes | 645.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'carpetas-error' on GitHub by visiting:
remote:   https://github.com/JBOrg10/JBGit/pull/new/carpetas-error
remote:
To https://github.com/JBOrg10/JBGit.git
 * [new branch]      carpetas-error -> carpetas-error
branch 'carpetas-error' set up to track 'origin/carpetas-error'.

```

Solucion carpetas y HTML #4

Merged JOELBUR19 merged 1 commit into `main` from `carpetas-error` now

Conversation 0 Commits 1 Checks 0 Files changed 4

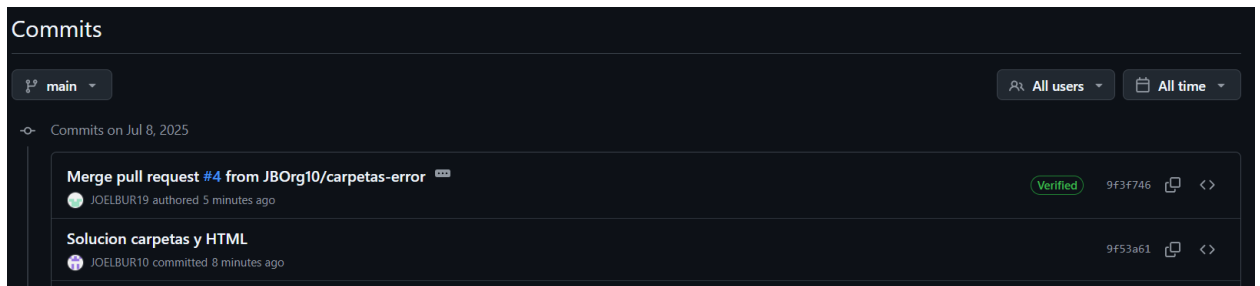
JOELBUR19 commented now Member ...

Actualización carpetas y bug archivo html.
Closes [#3](#)

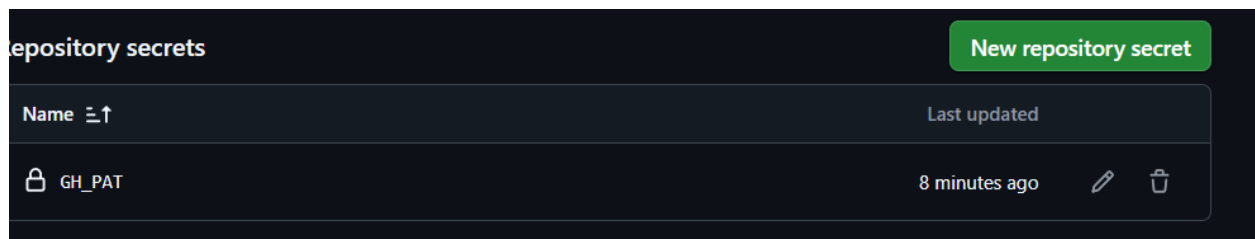
JOELBUR19 merged commit `9f3f746` into `main` now Revert

Pull request successfully merged and closed Delete branch
You're all set — the `carpetas-error` branch can be safely deleted.

JOELBUR19	Merge pull request #4 from JBOrg10/carpetas-error ...	9f3f746 · 1 minute ago	8 Commits
assets	Solucion carpetas y HTML	4 minutes ago	
documentation	Solucion carpetas y HTML	4 minutes ago	
src	Solucion carpetas y HTML	4 minutes ago	
LICENSE.txt	Modificación en Rama Pagina	46 minutes ago	
README.md	Update README.md	4 days ago	
REQUIREMENTS.md	Modificación en Rama Pagina	46 minutes ago	
backlog.md	Modificación en Rama Pagina	46 minutes ago	
index.html	Solucion carpetas y HTML	4 minutes ago	
styles.css	Modificación en Rama Pagina	46 minutes ago	

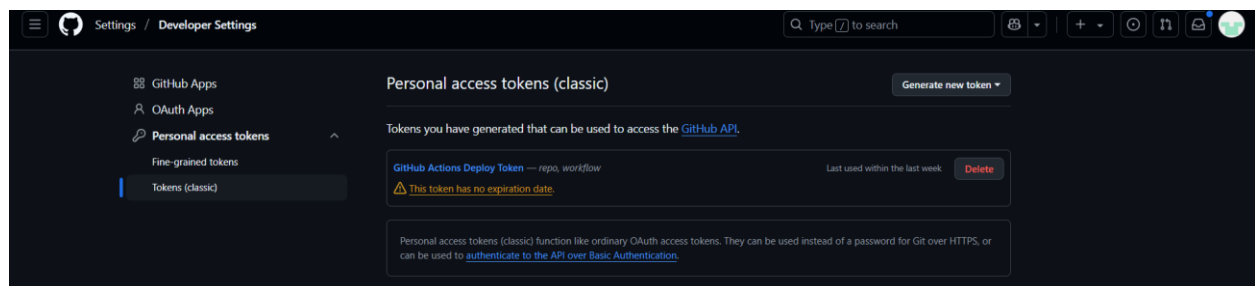


Actividad 4: Actions



Explicación funcionamiento Token

Tuve que crear un token secreto en mi cuenta de Github, crear una acción secreta en el repositorio de nombre GH_PAT, añadiendo ahí el token, el cual cuenta con permisos existentes para abrir y ver repositorios.



Continuación código .yaml

```
name: Montar mi pagina

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    steps:
      - uses: actions/checkout@v3
      - name: Validar HTML
        uses: Cyb3r-Jak3/html5validator-action@v7.2.0

  deploy:
    runs-on: ubuntu-latest
    if: github.event_name == 'push'
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to GitHub Pages
        uses: peaceiris/actions-gh-pages@v4
        with:
```

```
  with:
    github_token: ${{ secrets.GH_PAT }}
    publish_dir: ./
```

← Montar mi pagina

✓ Update montarencd.yml #5 Re-run all jobs

Summary

Jobs

- test
- ✓ deploy

Run details

- Usage
- Workflow file

Triggered via push 2 minutes ago

JOELBUR19 pushed · 83222fc · main

Status: Success

Total duration: 12s

Artifacts: —

montarencd.yml
on: push

- test 0s
- ✓ deploy 7s

GitHub Pages

GitHub Pages is designed to host your personal, organization, or project pages from a GitHub repository.

Your site is live at <https://jborg10.github.io/JBGit/>

Last deployed by JOELBUR19 13 minutes ago

[Visit site](https://jborg10.github.io/JBGit/)

Pagina funcionando en: <https://jborg10.github.io/JBGit/>

Referencias

Ceupe. (s.f.). *Componentes y tipos de bases de datos*. <https://acortar.link/vRNQh2>

Platzi. (s.f.). *Gestores de bases de datos*. <https://acortar.link/r68DIg>

Microsoft. (s.f.). *Tipos de datos*. <https://acortar.link/LfwfEn>

MongoDB Inc. (n.d.). Introduction to MongoDB. MongoDB Documentation.

<https://acortar.link/fWdZ1o>

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://acortar.link/gRHDfA>